Ordenamiento por Intercambio (Bubble Sort), del juego de tarjetas al código

Objetivo

Comprender el algoritmo de **Bubble Sort** mediante la simulación con tarjetas. Después, diseñar una solución en **pseudocódigo** e implementarla en **Java**.

Materiales

- Tarjetas con números o nombres (mínimo 5). Ejemplo: 34, 12, 25, 8, 19.
- Hojas para registrar las pasadas.
- Computadora con JDK instalado y Bloc de Notas

Fase 1: Simulación con tarjetas (se conserva de la actividad original)

1. Formen equipos de 5 o más personas. Cada integrante es una tarjeta y se colocan en fila.
2. Comparen pares adyacentes de izquierda a derecha. Si la tarjeta de la izquierda es mayor que la de la derecha, intercambian lugares.
3. Recorrer toda la fila es **una pasada**. Repitan las pasadas hasta que ya no haya intercambios.
4. **Registren en una tabla** el estado de la fila al terminar cada pasada, el número de comparaciones y el número de intercambios.

Fase 2: Propuesta de solución en pseudocódigo (en binas)

A partir del registro de la Fase 1, escriban su propio pseudocódigo. Debe responder a estas preguntas:

- ¿Qué estructura de datos representa la fila de tarjetas?
  Una lista doble enlazada
- ¿Qué ciclo representa las **pasadas** y cuál representa las **comparaciones** dentro de una pasada?
  Las pasadas representan un ciclo FOR y las comparaciones un IF
- ¿Cómo se realiza el **intercambio** de dos elementos sin perder uno de ellos?

```
Para i <- 1 Hasta n - 1 Hacer
  cambio <- Falso
  Para j <- 1 Hasta n - 1 - i + 1 Hacer
     Si arreglo[j] > arreglo[j + 1] Entonces
        temporal <- arreglo[j]
        arreglo[j] <- arreglo[j + 1]
        arreglo[j + 1] <- temporal
        cambio <- Verdadero
     FinSi

  FinPara
```

- ¿Cómo sabe el algoritmo que **ya terminó**, es decir, cuándo detenerse antes de completar todas las pasadas?
  Con una variable booleana que indica si se intercambió algún elemento. Si al realizar una pasada no se intercambia ningún elemento el arreglo ya está ordenado.

- ¿Cuántos elementos hay que comparar en cada pasada? Piensen en qué tarjetas ya quedaron en su lugar definitivo.
  Cada vez que se compara un elemento y se ordena ya no se vuelve a comparar entonces empieza a comparar desde el ultimo que ordeno y se van restando los que ya quedaron en su lugar.

Entregable: el pseudocódigo en hoja o archivo, con nombres de variables claros y sangría.

```
Algoritmo OrdenarArreglo

   Definir arreglo, n, i, j, temporal Como Entero
   Definir cambio Como Logico

   Dimension arreglo[5]
   arreglo[1] <- 8
   arreglo[2] <- 4
   arreglo[3] <- 6
   arreglo[4] <- 2
   arreglo[5] <- 1

         n <- 5
   Escribir "Arreglo original:"
   Para i <- 1 Hasta n Hacer
      Escribir Sin Saltar arreglo[i], " "
   FinPara

   Para i <- 1 Hasta n - 1 Hacer
      cambio <- Falso
      Para j <- 1 Hasta n - 1 - i + 1 Hacer
         Si arreglo[j] > arreglo[j + 1] Entonces
            temporal <- arreglo[j]
            arreglo[j] <- arreglo[j + 1]
            arreglo[j + 1] <- temporal
            cambio <- Verdadero
         FinSi
      FinPara
      Si cambio = Falso Entonces
         i <- n
      FinSi
   FinPara
   Escribir ""
   Escribir "Arreglo ordenado:"
   Para i <- 1 Hasta n Hacer
      Escribir Sin Saltar arreglo[i], " "
   FinPara

FinAlgoritmo
```

Fase 3: Implementación en Java

1. Traduzcan su pseudocódigo a un programa en Java con un método bubbleSort(int[] arreglo).
2. El programa debe mostrar el arreglo **después de cada pasada**, para compararlo con el registro de la Fase 1.
3. Debe contar e imprimir el **total de comparaciones, intercambios y pasadas**.
4. Prueben con el arreglo {34, 12, 25, 8, 19}, con uno ya ordenado y con uno en orden inverso.
5. Variación opcional: ordenar un arreglo de String alfabéticamente con compareTo.

Fase 4: Reflexión

- ¿Coincide la salida del programa con lo que registraron con las tarjetas?
- ¿Qué cambios tuvo que hacer su pseudocódigo al pasarlo a Java?
- ¿Qué pasa cuando el arreglo ya está ordenado? ¿Sigue comparando?
- ¿Es eficiente con listas muy grandes? Relacionen su respuesta con el número de comparaciones ($O(n^2)$).

Lista de cotejo (5 puntos)

#	Criterio	Cumple	No cumple	Observaciones
1	**Simulación y registro:** la tabla de pasadas con tarjetas es completa y correcta (estado de la fila, comparaciones e intercambios por pasada).			
2	**Pseudocódigo:** es claro, está estructurado y contiene los dos ciclos (pasadas y comparaciones), la condición de comparación, el intercambio con variable auxiliar y la condición de paro.			
3	**Implementación en Java:** el programa compila, ejecuta y ordena correctamente el arreglo de ejemplo y los casos de prueba (ordenado e inverso).			
4	**Coherencia pseudocódigo–código:** el programa respeta la lógica del pseudocódigo propuesto y muestra el arreglo tras cada pasada, junto con los contadores.			
5	**Reflexión y buenas prácticas:** responde las preguntas de reflexión con argumentos, incluida la eficiencia, y el código usa nombres significativos, sangría y comentarios.			